# PYTHON 101 MUAFİYET SINAVI

## "Topluluk Yönetim Sistemi" - Sınav Açıklamaları

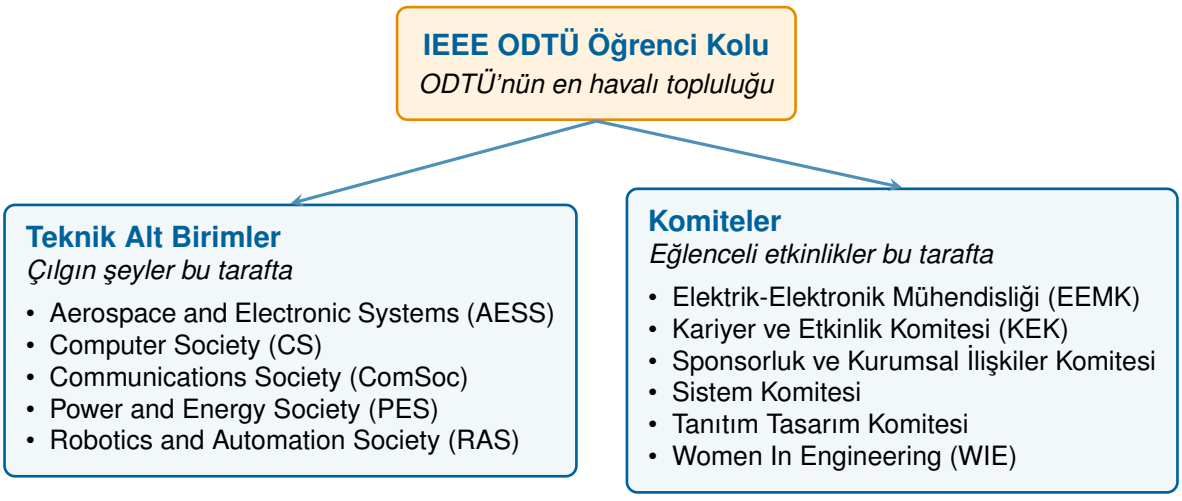
*Bu dosya sınav hakkındaki teknik detayları ve açıklamaları içerir.*

## 1. Senaryo: Topluluk Yönetim Sistemi

### 1.1 İşin arkaplanı

IEEE ODTÜ bu sene çok büyüdü, kurulduğu 1990'dan bugüne kadar 3 Teknik Alt Birimle yola çıkmışken, bu sene 36. yılında 2 yeni Teknik Alt Birim aramıza katıldı: ComSoc ve AESS. Hey, bir dakika! Ne, tüm alt birimleri bilmiyor musun? Onları tanıtmadan başlayamam ki! Hadi gel önce IEEE ODTÜ'yü tanıyalım!

O kadar çok alt birim var ki (hepsine katılman denemen lazım!), bizler alt birimlerimizi ikiye ayırdık: Teknik Alt Birimler ve Komiteler. Teknik Alt Birimler globalde IEEE altında bulunan 39 topluluk içerisinden oluşurken Komiteler bize özel, sosyal alt birimlerimizdir (Sadece WIE ve Sistem Komitesi global IEEE'de yer almaktadır.).




Bu kadar alt birim varken, hele ki tüm 5 Teknik Alt Birimler haftalık aktivite/eğitim yapmak isterken bu topluluğu idare etmek kolay mı? Tabii ki de hayır! Teknik YK'mız Damla bu durum üzerine çıldırdı ve gece 3.00'te CS'i aradı (zaten CS uyumadığı için hemen açtık telefonu) ve "Artık bir yönetim sistemine ihtiyacımız var!" (yarı-tehditkar) dedi. Biz de fırsat bu fırsat neden siz yapmıyorsunuz dedik (kesinlikle biz yapamadığımız için değil)! İşte karşınızda gerçek bir muafiyet sınavı: Beraber topluluk yönetim sistemi yapıyoruz!

### 1.2 Aktif üye yönetim sistemi

Aktif üyeler bizim her şeyimiz, canımız ciğerimiz. Peki, aktif üye neydi, nasıl olunuyordu? Her Teknik Alt Birim, birtakım eğitimler verir: AESS Aviyonik ve Mekanik, CS Python, ComSoc MATLAB, PES Enerji ve Araştırma, RAS C dersi verir. Bu eğitimleri geçenler aktif üye olur ve bizimle beraber oldukça çılgın ve bir o kadar da fenasal işlere imza atar. Bu eğitimleri bilenler ise muafiyet sınavı aracılığı ile aktif üye olabilir, tam da şimdi olduğu gibi!

Bu kadar aktif üyeye sahip olunca (en son saymadım ama baya bi aktif üye vardı, 200 civarı falan), onları takımlara yerleştirirken de dikkatli olmak lazım, herkesi yeteneğine göre yerleştirmek ve kimseyi çok fazla takımda yormamak lazım! Dolayısıyla bizler bir sistem hazırladık, ancak gece 3.00'te başlayınca sabaha doğru uyuyakalmışım (gözlerimi dinlendriyorum), dolayısıyla bazı takım bilgileri ve kuralları yarım kalmış, onları tamamlaman lazım!

Aktif üyeleri takımlara alırken yalnız sakın unutma, burası IEEE ODTÜ, burada bir Teknik Alt Birimin aktif üyesi diğer Teknik Alt Birimlerin takımlarında yer alabilir! Biz böylece enerjik ve güçlü takımlar üretiyoruz, takımlar böyle olunca İç İlişkiler YK'mız Nisa çok mutlu oluyor, onu üzmeyelim bence.

Sıra en önemli kısma geldi. Bu sene Başkanımız Erenalp ve Dış İşleri YK'mız İzzet kasanın kapısını açtı (keşkeee), her takıma bütçe ayarlıyorlar, hatta kalabalık ve multidisipliner takımlara daha çok bütçe ayrıldığına dair söylentiler var! Bunlar için de İzzet'e yardım etmemiz lazım unutmayalım. Her takım için bütçe hesaplayıp, büyük ve karma takımları da belirleyip raporlamamız lazım.

### 1.3 Takvim yönetim sistemi

Her Teknik Alt Birim haftada bir gün ders/etkinlik yapmak isterse tüm hafta bitiyor (bazıları var biliyorum tüm hafta geliyor, cansınız). Bunları takvime yerleştirmemiz lazım, bazen bir alt birimin istediği gün diğeri ile çakıştığı zaman olabilir, bunları engellemeliyiz. Çakışmayan etkinlikleri takvime yerleştirmek lazım. Bunun için çalışmalara tam başlayacaktım ama muafiyet sınavını hazırlamam gerektiği için yarım kaldı. Etkinlik sisteminin temelini yaptım ama gerisi sende!

Bir de, bazen beraber etkinlik yaptığımız oluyor, onlara Nisa daha çok kıymet veriyor (neymiş, daha keyifli oluyormuş efendim de öyle de etkinlik lazımmış) o yüzden onlar öncelikli. Sakın unutma, sana güvenim tam!

Burada tüm hikayeyi anlattım, artık devamı sende, ihtiyacın olan tüm bilgiler bu belgenin devamında yer alıyor, iyi şanslar!

## 2. Sınav Kuralları ve İşleyiş

### 2.1 Sınava Nasıl Başlarım?

Sakin ol şampiyon, sınava başlayacaksın, hadi gel nasıl başlayacağımızı öğrenelim.

1. Öncelikle buraya tıklayıp Sınavın olduğu GitHub repo'suna gitmen lazım. Burada karşına bir *README.md* dosyası sana aşamaları anlatacak, ama okumayacağını biliyorum o yüzden bir daha açıklamam lazım maalesef.
2. En üstte yer alan birkaç sekme var, `Code` yazan sekmenin hemen yanında `Issues` diye bir sekme var, sakince oraya tıkla.
3. Üst sağda mavi renkli `New Issue` diye bir buton var, ona tıkla. Karşına çıkan seçenekler arasında `Blank Issue` seçeneğini seç.
4. *Title* olarak `Start Exam` yaz (lütfen büyük ve küçük harflere dikkat edelim) ve `Create` butonuna bas, başka bir açıklama filan yazmana hiç gerek yok.
5. Sonrasında 10 sn beklemen yeterli (dikkat süren 15 sn o yüzden bence gayet okey), sonrasında sayfayı yenileyince bir otomatik mesaj göreceksin!

6. Lütfen öncelikle sana attığım daveti kabul et, etmezsen başlayamazsın maalesef. Burası çok önemli. Daveti kabul ettikten sonra senin için oluşturduğum repo'ya geçebilirsin ve artık her şey senin elinde!
7. İstersen *GitHub Codespaces* ile online istersen de lokal olarak repo üzerinde çalışabilirsin!

## 2.2 Sınav İşleyişi

Sınava başlamadan önce bazı kuralların var olduğunu hatırlatmam lazım. Merak etme, basit kurallar var, onlara uyarsan sınavı geçmen çok kolay.

- Zaman kısıtlaması yok, relax relax çöz, bizde darlama yok (ama ne kadar erken aramıza katılırsan o kadar çok eğleniriz!). Ancak bir puanlama kısıtlaması mevcut. Sadece 3 kez push yapma hakkınız var, bu 3 submission içerisinden en yüksek puanınız sizin notunuzu oluşturmakta. 4. ve daha sonraki push'lar hesaba alınmaz.
- Sınav için herhangi bir geçme notu söz konusu değil, bunu sınavın yapma oranına göre ben belirleyeceğim (gene ODTÜ gene curve), sonuçları ona göre duyuracağım.
- Sınavda bu 3 submission hakkından dolayı güzel bir kıyak geçtim. Lokal olarak ufak bir test yapabilirsiniz! Sadece terminal'de `python exam/local_tests.py` yazıp çalıştırmanız yeterli olacaktır. Uç durumları (edge case) içermese de kodunuzdaki basit hataları hızlıca tespit edip size haberdar eder.
- Sınavda 5 soru olup soru başına 5 adet *case* bulunmaktadır, toplam 25 *case* söz konusu. Ancak bunlar gizli ve sadece siz push yaptığınızda puanlama için kullanılıyor. Dolayısıyla kodu push'lamadan önce mutlaka lokal testin hata vermediğinden emin olun.
- Toplamda 5 soru var demiştim, bunları A ve B olarak iki kısma ayırdım, ilk 3 soru part A, son iki soru ise part B olarak ayrılmış durumda. Ek olarak tüm soruların test case'leri birbirinden bağımsız, bu sayede bir soruyu yapamadığınız takdirde diğer sorulardan asla puan kaybetmeyeceksiniz! Bu da ikinci kıyağımdı (CENG'de bize bu kıyağı geçmiyorlar düşünün) aklınızda bulunsun.
- İki tane dosya *read_only* ifadesi ile başlıyor: `read_only_base.py` ve `read_only_helpers.py`. Bu dosyaları herhangi bir şekilde editlerseniz sistem bunu tespit eder ve uyarı verir! Lütfen böyle maceralara kalkışmayalım.
- Bir de, yapay zeka kullanımı yasak, kodu siz yazmıyorsanız anlamı yok tabii ki. Ben yazdığınız her kodu tek tek okuyacağım (evet okurum), ve yapay zeka olanları tespit edersem (ki ederim) notunu direkt 0 vereceğim, aklınızda bulunsun.

*NOT: Sakın unutma, commit yapmanda sorun yokken yaptığın her push işlemi direkt notunun değerlendirilmesi anlamına gelir!*

# 3. *Read-Only* Temel ve Yardımcı Kodlar

## 3.1 *read_only_base.py*

Bu dosyada bayağı bir şey var, hadi beraber inceleyelim!

- ```python
  class Society(Enum):
      """..."""

      CS = "CS"
      COMSOC = "ComSoc"
      RAS = "RAS"
      PES = "PES"
      AESS = "AESS"
  ```

  Bu class basit bir *enumeration* class'ı. Bu class ile beraber standard bir şekilde Teknik Alt Birim isimlerini kullanmanız gerekmektedir. Burada şunu unutmayınız ki, eğer bir objeye erişmek istiyorsanız örn. `Society.CS`, eğer onun string değerine erişmek istiyorsanız örn. `Society.CS.value` kullanmanız gerekmektedir.

- ```python
  class ActiveMember:
  """..."""

  def __init__(
      self, student_id: str, name: str, home_society: Society, skills: set[str]
  ):
      self.student_id: str = student_id
      self.name: str = name
      self._home_society: None | Society = None
      self.home_society: Society = home_society
      self.skills: set[str] = set(skills)
      self._team_count: int = 0

  ...

  def __str__(self) -> str:
      return f"{self.name} ({self.student_id}) - {self.home_society.value}"
  ```

  Bu ise bizim aktif üye class'ı. Her aktif üyenin öğrenci numarası, adı, aktif üyesi olduğu Teknik Alt Birim, sahip olduğu yetenekleri ve bulunduğu takım sayısı kayıtlı.

- ```python
  class BaseProjectTeam:
  """..."""

  def __init__(
      self,
      team_name: str,
      owner_society: Society,
      max_capacity: int,
      required_skills: set[str],
  ):
      self.team_name: str = team_name
      self._owner_society: None | Society = None
      self.owner_society: Society = owner_society
      self._max_capacity: None | int = None
      self.max_capacity: int = max_capacity
      self.required_skills: set[str] = set(required_skills)
      self.roster: list[ActiveMember] = []

  ...

  def __str__(self) -> str:
      return (
          f"{self.team_name} [{self.owner_society.value}] "
          f"({len(self.roster)}/{self.max_capacity})"
      )
  ```
  ```

Proje takımı class'ı da bu şekilde. IEEE ODTÜ'de METUROİD, METU Ch-ARGE gibi çok büyük takımlar var, onları da tabii ki bu kadar basit tutmak mümkün değil, bu class'ı *inherit* edip kullanacağız. Her takımın gördüğünüz gibi adı, altında bulunduğu Teknik Alt Birim, maksimum kapasitesi, gereken beceriler ve üye listesi yer almakta.

- ```
  class EventProposal:
  """..."""

      def __init__(
          self,
          event_id: str,
          title: str,
          day: str,
          start_min: int,
          end_min: int,
          hosting_societies: set[Society],
      ):
          self.event_id: str = event_id
          self.title: str = title
          self.day: str = day
          self.start_min: int = start_min
          self.end_min: int = end_min
          self.hosting_societies: set[Society] = set(hosting_societies)

      ...

      def __str__(self) -> str:
          return (
              f"{self.event_id}: {self.title} "
              f"({self.day} {self.start_min}-{self.end_min})"
          )
  ```

Karşınızda aktivite önerisi class'ı. Her aktivite önerisi bir akivite numarasına, başlığa, güne (MON, WED, FRI gibi 3 karakterli büyük harflerle), başlama ve bitiş dakikalarına (gece yarısından başlamak üzere toplam dakika sayısı, bu konuda unix timestamp sisteminden örnek aldım, ne olduğunu öğrenmek için buraya tıklayabilirsiniz.) ve organize eden Teknik Alt Birim ya da Birimlere sahiptir.

### 3.2 *read_only_helpers.py*

Bu dosyada ise iki tane yardımcı fonksiyon bulunmakta, hadi beraber inceleyelim!

- ```
  def calculate_base_grant(member_count: int, unique_skill_count: int) -> float:
      if member_count <= 0:
          return 0.0
      return (member_count * 500.0) + (unique_skill_count * 250.0)
  ```

Bu fonksiyon İzzet'e yardım etmesi için özel olarak tasarlandı, ama dikkat, multidisipliner takımları hesaba katmıyor burası!

- ```
  def is_overlapping(
      event_a_start: int, event_a_finish: int, event_b_start: int, event_b_finish: int
  ) -> bool:
      return event_a_start < event_b_finish and event_b_start < event_a_finish
  ```

Bu fonksiyon da çakışan etkinlikleri tespit edip Damla'ya yardım ediyor, böylece bir daha gece 3.00'te aranmama gerek kalmayacak!
  ```

# 4. Part A: Takım Sistemi ve Bütçelendirme (60 Puan)

## 4.1 Soru 1: SocietyTeam & Topluluk Class'ları (20 Puan)

**Hedef Dosya:** `exam/student_classes.py`

Bu soruda tam olarak 6 adet class tanımlamanız gerekmekte. Öncelikle ilk class'ı tanıyarak başlayalım.

`SocietyTeam` class'ı `BaseProjectTeam` class'ını *inherit* eder. Bu class'ı kurduktan sonra iki şey eklemeniz gerekmekte:

- **`def is_interdisciplinary(self) -> bool:`** Bu *attribute* `property` ile tanımlanmalıdır. Bir takımda eğer en az 2 kişi varsa, en az bir kişi takımın ait olduğu Teknik Alt Birimin aktif üyesiyse ve en az bir kişi başka bir Teknik Alt Birimin üyesiyse o takım **multidisipliner** olarak kabul edilir.
- **`def can_recruit(self, member: ActiveMember) -> bool:`** Bu fonksiyon ise verilen bir aktif üyenin o takımda yer alıp alamayacağını geri döner. Öncelikle fonksyion girdinin bir aktif üye oladuğunu doğrulamalıdır. Sonrasında takımda hala yer varsa, söz konusu aktif üye takımda zaten yer almıyorsa, üyenin yetenekleri takımda gereken yeteneklerle en az iki ortak yeteneğe sahip olmalıdır. Ek olarak bir üye en fazla iki takımda yer alabilir. Şartları sağlayan üyeler için fonksiyon *True* döner.

Geriye kalan diğer 5 class ise her bir Teknik Alt Birim bünyesindeki takımları temsil eder. Bu takımlar `SocietyTeam` class'ını *inherit* Onları tanımlarken bazı verileri sabit bazılarını ise *default argument* olarak belirtmeniz gerekmektedir. Özet bilgiye aşağıdaki tablo üzerinden erişebilirsiniz.

| **Subclass Name** | `owner_society` | `max_capacity` | **Default** `required_skills` **Set** |
|---|---|---|---|
| `CSTeam` | `Society.CS` | 6 | `{"Python", "C++", "AI", "Git"}` |
| `ComSocTeam` | `Society.COMSOC` | 8 | `{"MATLAB", "SDR", "RF", "Signal Processing"}` |
| `RASTeam` | `Society.RAS` | 10 | `{"C", "ROS", "Control", "Arduino"}` |
| `PESTeam` | `Society.PES` | 5 | `{"Power Systems", "Simulink", "BMS", "Python"}` |
| `AESSTeam` | `Society.AESS` | 15 | `{"Python", "CAD", "Avionics", "GNC"}` |

### 4.2 Soru 2: Aktif Üye Alımları (20 Puan)

**Hedef Dosya:** `exam/student_functions.py`

**Fonksiyon:**

```
def draft_active_members(
    candidates: list[ActiveMember], teams: list[SocietyTeam]
) -> dict[str, list[str]]:
```

Bu fonksiyon içerisinde her takımı tek tek işlemeniz lazım. Öncelikle `.can_recruit()` fonksiyonundan da yardım alarak takıma alınmaya uygun üyeleri filtreleyebilirsiniz. Sonrasında bu üyeleri şu öncelik sırası ile sıralamanız gerekmekte: **1.** Takım ile en çok uyuşan beceriye sahip üyeler daha önceliklidir, **2.** Daha az takımda görev alan üyeler daha önceliklidir, **3.** Eşitlik durumlarında ise en son öğrenci numarasına alfabetik olarak bakılır. Sıralanmış üyeler arasında sırası ile alabildiğimiz üyeleri (`.can_recruit()` ile tekrar kontrol edin, çünkü takım dolmuş olabilir) *roster*'a ekleyip üyenin takım sayısını güncellemeniz gerekmektedir. En son çıktı olarak `{takım adı: [takım üyelerinin öğrenci numaraları]}` şeklinde bir *dict* dönmelidir.

### 4.3 Soru 3: Takım Enerjisi & Bütçe Hesaplama (20 Puan)

**Hedef Dosya:** `exam/student_functions.py`

**Fonksiyon:**

```
def audit_branch_synergy(teams: list[SocietyTeam]) -> dict:
```

Bu soruda her takım için takımda yer alam üyelerin yeteneklerini bir set içerisinde bir araya getirmeniz gerekmektedir. Bunu yaparken Python'ın set *union* işlemini kullanabilirsiniz. Bu elde ettiğiniz set'in uzunluğu ve üye sayısını sizlere verdiğimiz yardımcı fonksiyona atmanız ve eğer takım multidisipliner ise elde ettiğiniz sonucu **1.25** ile çarpmanız gerekmektedir. Bu bütçeyi tüm takımlar için toplamanız gerekmekte. Sonrasında takımlar arasında büyük ve karma olanları tespit etmeniz gerekmekte. Bir takımda en az 3 Teknik Alt Birim üyesi varsa ve takım ya dolu ya da maksimumdan 1 kişi eksik ise o takım büyük ve karma takım kabul edilir. En son ise her bir Teknik Alt Birimde kaç tane üye olduğunu bulmanız lazım. Burada ise bir üyenin birden fazla takımda olabildiği için bazı üyeleri çift saymamaya dikkat etmeniz de lazım. İstenen çıktı şu formatta:

```
return {
    "Total Budget": toplam bütçe,
    "Huge Mixed Teams": [takım isimleri],
    "Society Members Count": {"CS": 0, "ComSoc": 0, "RAS": 0, "PES": 0, "AESS": 0},
}
```

# 5. Part B: Etkinlik & Takvim Yönetimi Sistemi (40 Puan)

## 5.1 Question 4: Event Proposal Parser & Validator (20 Points)

**Hedef Dosya:** `exam/student_functions.py`

**Fonksiyon:**

```
def parse_event_proposals(raw_lines: list[str]) -> list[EventProposal]:
```

Burada gelen veri `"EVT-101 | CS, AESS | WED | 17:30-19:00 | QT Yer İstasyonu Workshop"` örneğinde olduğu gibi bir formatta gelmektedir. Listedeki her etkinlik için bu format geçerlidir: **1.** Etkinlik numarası, **2.** Teknik Alt Birim ya da Birimler, **3.** Gün (gene aynı 3 harfli büyük harflerle), **4.** Başlangıç ve Bitiş Saatleri (dakikaya çevirmeyi unutmayın), **5.** Takım İsmi. Tüm etkinlikleri `EventProposal` olarak kaydedip bir liste olarak dönmeniz gerekmektedir.

Ek olarak bu fonksiyonun şu üç durumda `ValueError` döndürmesi gerekmektedir: **1.** Gün verisi haftanın 7 gününden biri olmalıdır, **2.** Teknik Alt Birim isimleri 5 Teknik Alt Birim ile eşleşmelidir, **3.** Etkinliğin bitiş saati başlama saatiyle aynı ya da önce olmamalıdır ve 90 dakikayı geçmemelidir.

## 5.2 Soru 5: Weekly Auditorium Conflict Resolver (20 Points)

**Hedef Dosya:** `exam/student_functions.py`

**Fonksiyon:**

```
def build_weekly_calendar(proposals: list[EventProposal]) -> dict[str, list[str]]:
```

Bu soruda ise her günün bulunduğu `{gün: [etkinlik numaraları]` formatında bir *dict* dönmeniz gerekmekte. Öncelikle her gün için etkinlikleri tek tek kategorize etmek lazım. Sonrasında şu öncelik sırasına göre her gün için etkinlikler sırlanmalı: **1.** Ne kadar çok Teknik Alt Birim beraber ise o kadar önceliklidir, **2.** En erken başlayan etkinlikler daha önceliklidir, **3.** Beraberlik durumlarında ise etkinlik numarası alfabetik olarak sıralanır. Sıralanmış aktiviteleri eğer kabul edilmiş aktiviteler ile çakışmıyorsa (bunu size sağladığımız yardımcı fonksiyon ile kontrol edebilirsiniz) sırasıyla kabul edilmiş aktivitelere eklemeniz gerekmekte. Kabul edilen tüm aktiviteleri kronolojik olarak sıralayıp etkinlik numaralarını liste şeklinde *dict*'e kaydetmeniz gerekmekte.

---

*Baya bir bilgi verdim, artık IEEE ODTÜ'nün kaderi senin ellerinde!*
*Sınavı tamamla ve aramıza katıl! Başarılar! – IEEE ODTÜ CS Başkanı Ahmet Tağluk*